

Aula 4 - Dockerfiles Pro

Docupedia Export

Author:Cardoso Cristian (CtP/ETS)

Date:29-May-2026 18:13

Table of Contents

1	DockerHub	3
1.1	Criou uma nova conta.	3
1.2	Login com Github	3
1.3	Como publicar uma imagem	5
2	Problema	6
2.1	Hora do pensamento I	6
2.1.1	Conclusão	6
2.2	Hora do pensamento II	6
2.3	Conclusão	6
2.4	Conclusão do problema	6
3	Solução	9
3.1	Como fazer	10
3.2	Resultado	11
4	Teste do resultado de Mult-Stage	12
4.1	DockerHub	13
5	Debian VS Alpine	14
5.1	Aplicando Alpine	14
5.2	DockerHub	15
6	Rootless	16
7	Atividade	18

1 DockerHub

Vamos criar uma conta no [DockerHub](#), o GitHub de imagens do docker.

1.1 Criou uma nova conta.

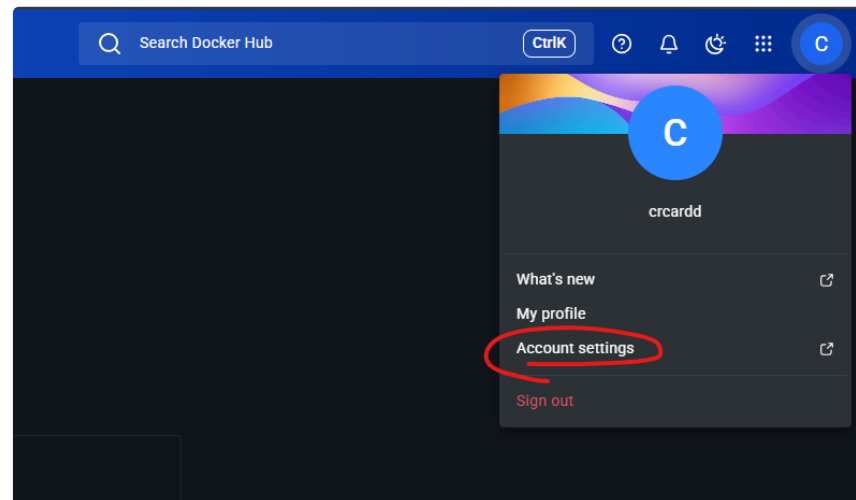
- No terminal powershell do seu computador execute:

```
docker login
```

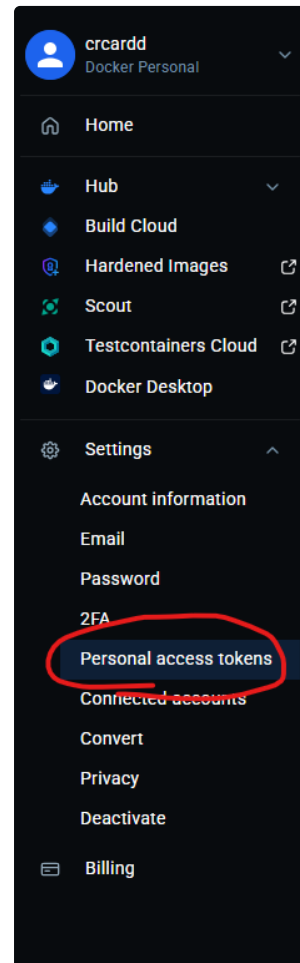
- Coloque seu nome de usuário
- Ao pedir sua senha cole o token mostrado na página do navegador

1.2 Login com Github

- Acesse as configurações da sua conta:



- Acesse "**Personal Token Access**"



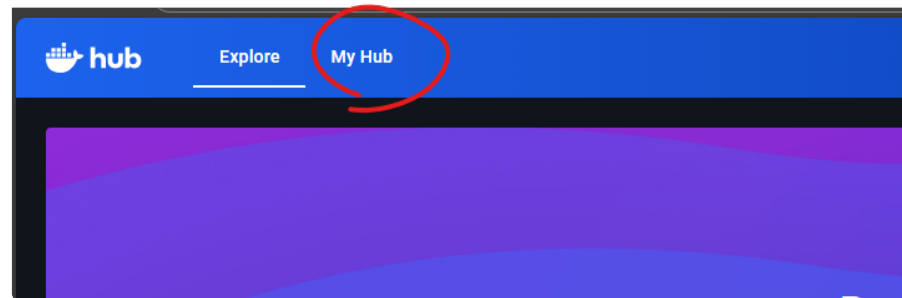
- Aperte em "**Generate new Token**"
- Adicione uma descrição ao token (ex: AulaDeDocker)
- Altera a opção de "Access Permissions", para que possamos publicar nossa imagem coloque "**Read & Write**"
- No terminal powershell do seu computador execute:

```
docker login
```

- Coloque seu nome de usuário
- Ao pedir sua senha cole o token mostrado na página do navegador

1.3 Como publicar uma imagem

- Vá em My Hub



- Crie um repositório
- Agora dentro do powershell do seu computador digite o seguinte comando:

```
docker images
```

- Pegue o ID da sua imagem do **client** (imagem criada na aula anterior)
- Execute o seguinte comando:

```
docker tag <ID_DA_IMAGEM> <NOME_DE_USUARIO_DO_DOCKERHUB>:<NOME_DO_REPOSITARIO_CRIADO>:v1
```

- E em sequência rode:

```
docker push <NOME_DE_USUARIO_DO_DOCKERHUB>:<NOME_DO_REPOSITARIO_CRIADO>:v1
```

****Observe o tempo que irá levar para publicarmos esta imagem e guarde isso**

2 Problema

Na aula anterior utilizamos mais de 10 containers para nosso laboratório para experimentar a distribuição de pedidos e o balanceamento de carga. Mas da maneira que foi feita isso gerou um grande problema, o código em questão não é nada mais que um looping com uma requisição e uma impressão no terminal. Mas podemos observar o impacto do que fizemos da seguinte forma:

Execute o seguinte comando no terminal:

```
docker images
```

Isso irá mostrar no terminal o tamanho de cada imagem presente na sua máquina, procure a imagem criada na aula passada e observe o tamanho absurdo dela:



IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
client:latest	d257a9768dcf	1.2GB	318MB	U
crcardd/plankton:latest	dfa6f917322e	305MB	65.9MB	U
hashicorp/http-echo:latest	fc75f691c8b	16.7MB	4.63MB	U
nginx:latest	bc45d248c4e1	240MB	65.8MB	U

2.1 Hora do pensamento I

Se nossa imagem pesa 1.2GB, isso significa que estamos utilizando aproximadamente 12GB?

2.1.1 Conclusão

[RESPOSTA AQUI DEPOIS]

2.2 Hora do pensamento II

Por que isso acontece?

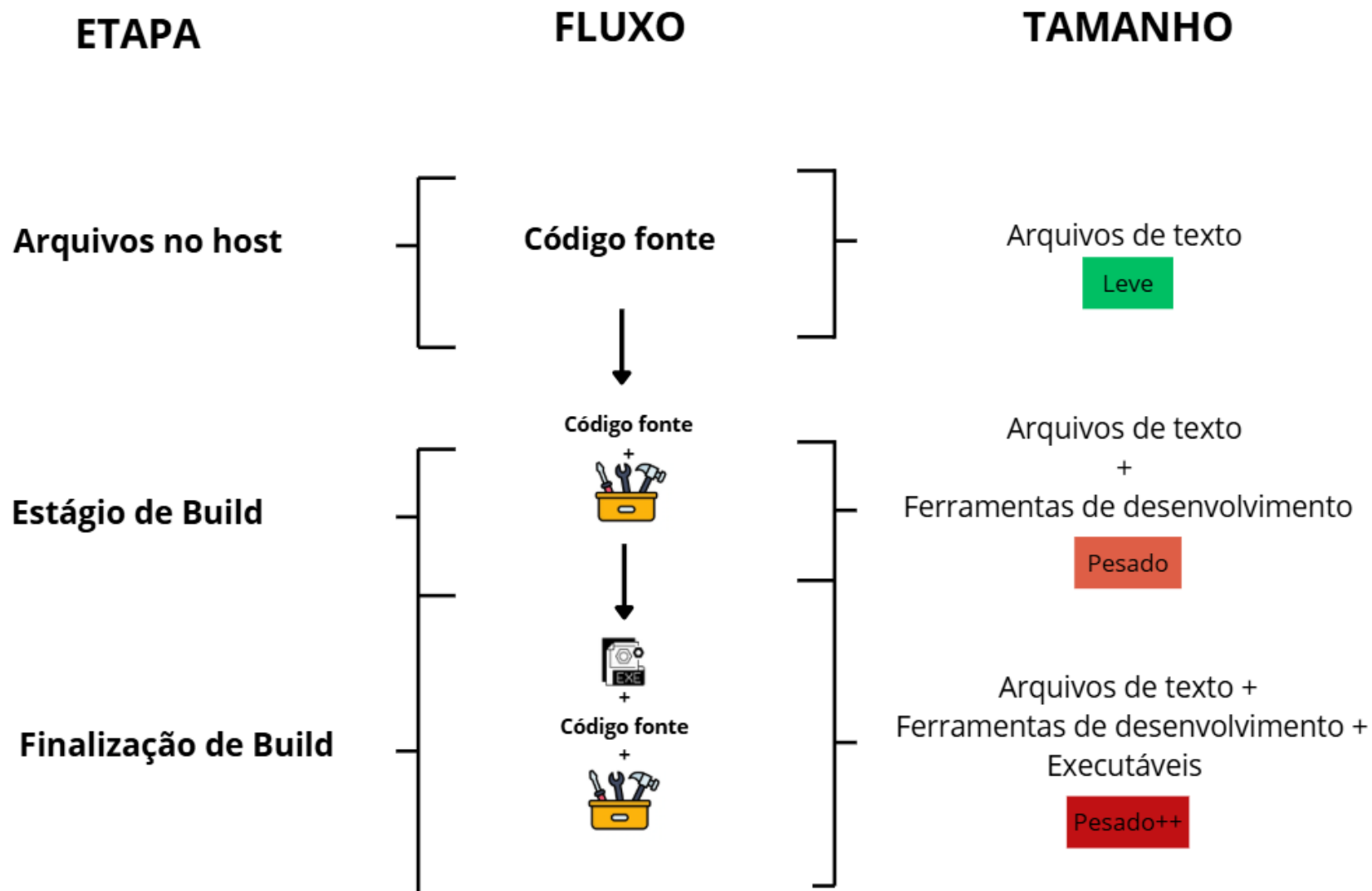
2.3 Conclusão

[RESPOSTA AQUI DEPOIS]

2.4 Conclusão do problema

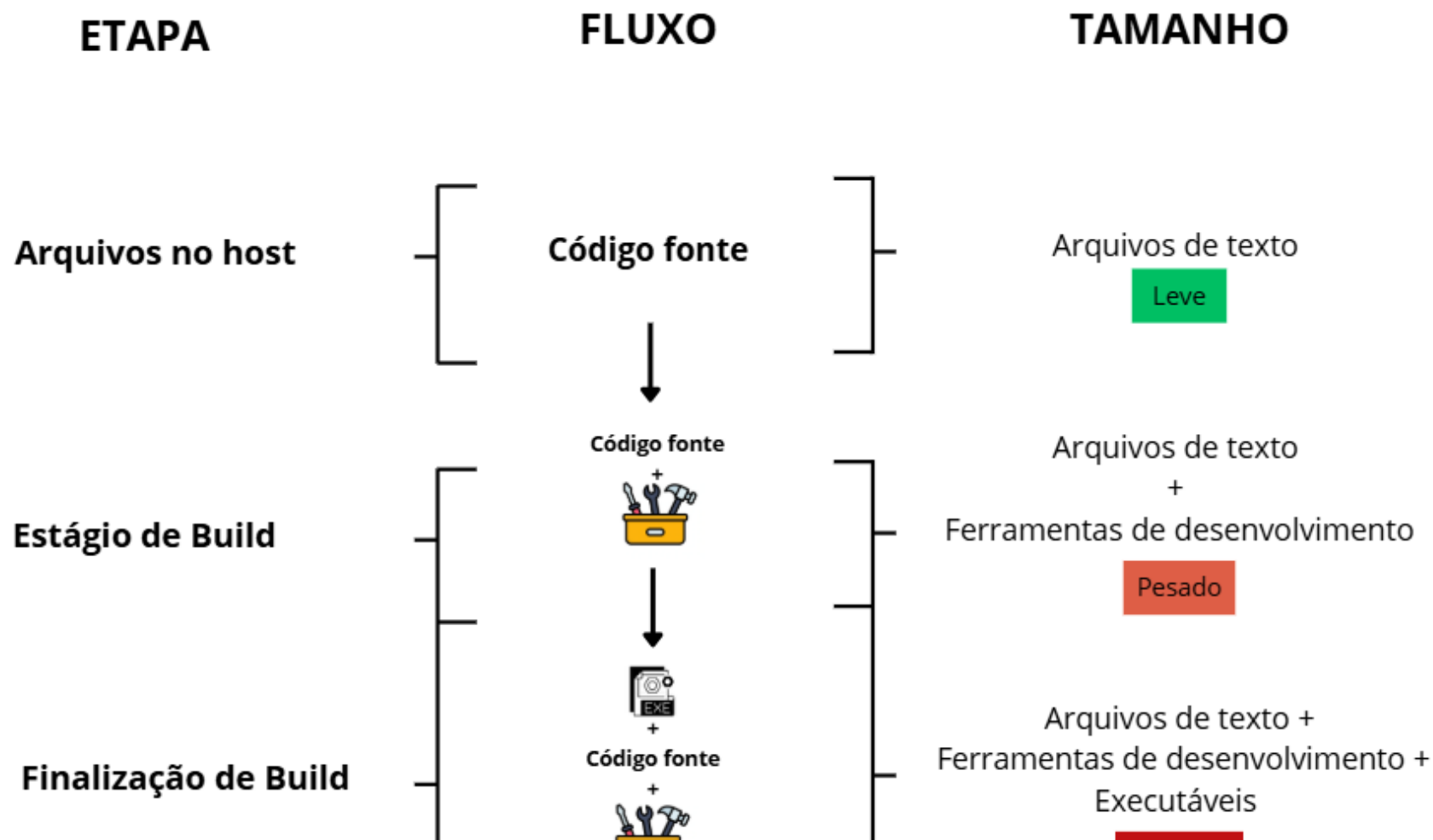
Nós não precisamos do poder todo da linguagem escolhida para **executar** o sistema. Para o desenvolvimento é comum usarmos os kits de ferramentas (SDK, JDK). Porém assim que o programa está pronto não precisamos deles mais, e se tornam peso morto no nosso container, eles são úteis apenas para transformar o código em

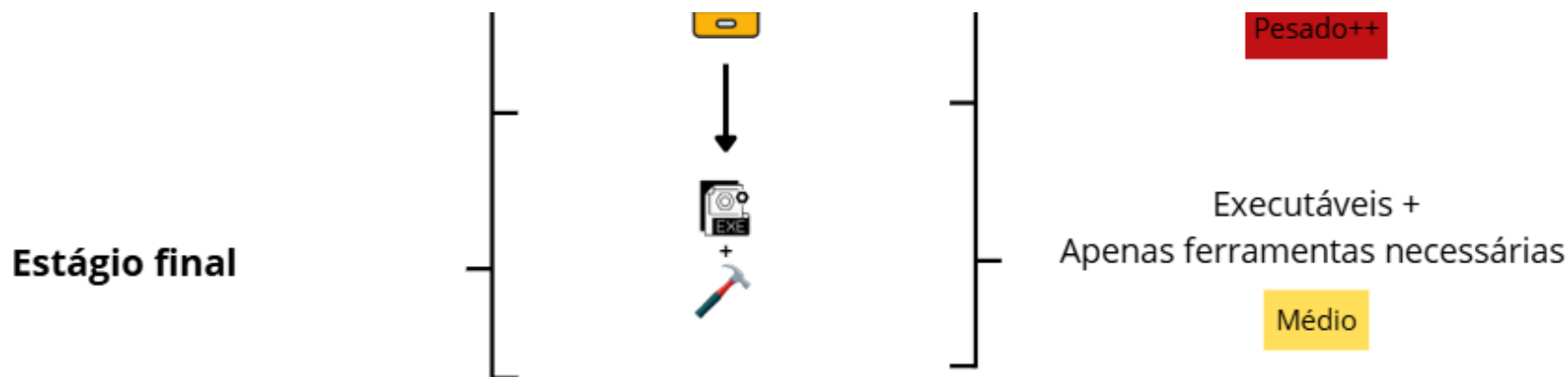
programa, mas assim que todo o código está pronto podemos e devemos publicar nosso código, ou seja, buildar ele para ele estar pronto a todo momento. Em produção, devemos utilizar apenas o **Runtime** (o ambiente mínimo de execução), garantindo que o container seja leve, rápido e contenha apenas o estritamente necessário para o sistema funcionar.



3 Solução

Para resolver este problema podemos fazer de duas maneiras, podemos publicar nosso projeto na nossa máquina, o que implica em termos as ferramentas necessárias já baixadas na nossa máquina. Ou então podemos utilizar uma técnica chamada **Mult-Stage-Building**, que serve para temporariamente termos todo o ambiente de desenvolvimento no container, publicarmos nosso sistema, e então remover o ambiente de desenvolvimento e baixarmos o ambiente de produção. Esquemmatizando o fluxo:





3.1 Como fazer

Para isso precisamos importar duas imagens no nosso Dockerfile em momentos diferentes, como é possível perceber pela imagem, iniciamos com uma imagem com todo o kit de ferramentas, após transformarmos nosso código fonte em arquivos brutos (.exe), podemos passar a usar uma imagem que contenha apenas o necessário para executar aqueles arquivos, mas devemos ter cuidado, pois nessa troca de imagens é importante se assegurar que os devidos arquivos não serão perdidos, vamos a demonstração para melhor entendimento:

Aqui iremos realizar mult-stage com um serviço .NET:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /app]

COPY ./CODIGO/*.csproj ./
RUN dotnet restore
COPY ./CODIGO ./
```

Até aqui está tudo normal, conforme o que temos visto nas aulas passadas, mas agora iremos iniciar o que faz com que tudo funcione.

Primeiro iremos publicar nosso código para ser compilado em um único arquivo:

```
RUN dotnet publish -c Release -o /app/publish
```

Agora iniciaremos uma imagem com os recursos mínimos necessários para rodar nosso código

```
FROM mcr.microsoft.com/dotnet/aspnet:9.0
```

```
WORKDIR /app
```

E então chegamos na parte que conecta ambas as imagens, pois podemos acessar o que foi gerado na primeira imagem a partir da segunda da forma demonstrada abaixo, com isso copiamos os arquivos compilados do nosso serviço para a nossa pata atual

```
COPY --from=build /app/publish .
```

E executamos deixamos programado para ser executado da seguinte maneira:

```
ENTRYPOINT ["dotnet", "CODIGO.dll"]
```

3.2 Resultado

```
# Etapa 1
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /app

COPY ./CODIGO/*.csproj ./
RUN dotnet restore
COPY ./CODIGO ./

RUN dotnet publish -c Release -o /app/publish

# Etapa 2
FROM mcr.microsoft.com/dotnet/aspnet:9.0

WORKDIR /app
COPY --from=build /app/publish ./

ENTRYPOINT ["dotnet", "CODIGO.dll"]
```

4 Teste do resultado de Mult-Stage

Utilize a seguinte imagem e builde ela:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /app

COPY ./CODIGO/*.csproj ./
RUN dotnet restore
COPY ./CODIGO ./

ENTRYPOINT ["dotnet", "run"]
```

Execute o comando:

```
docker build -t codigopesado:latest ./
```

Resultado:

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
codigopesado:latest	ec3b3395e8a2	1.2GB	318MB	

Utilize a seguinte imagem e builde ela:

```
# Etapa 1
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /app

COPY ./CODIGO/*.csproj ./
RUN dotnet restore
COPY ./CODIGO ./

RUN dotnet publish -c Release -o /app/publish

# Etapa 2
FROM mcr.microsoft.com/dotnet/aspnet:9.0

WORKDIR /app
COPY --from=build /app/publish ./
```

```
ENTRYPOINT ["dotnet", "CODIGO.dll"]
```

Execute o comando:

```
docker build -t codigootimizado:latest ./
```

Resultado:

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
codigootimizado:latest	d80c8a10d557	329MB	92.9MB	
codigopesado:latest	ec3b3395e8a2	1.2GB	318MB	

E agora podemos observar a diferença, nos livramos de quase 900MB de uso do disco para manter esse ambiente, e de mais de 200MB de conteúdo dos arquivos do container.

Vale lembrar que é sempre muito importante ter um **.dockerignore** para agilizarmos a primeira etapa do build, evitando a cópia de arquivos desnecessários e ainda pesados podemos aumentar a eficiência para subir nossas aplicações.

4.1 DockerHub

Publique a primeira no seu repositório do **DockerHub** como a **:v1**

Publique a segunda no seu repositório do **DockerHub** como a **:v2**

5 Debian VS Alpine

O sistema padrão das imagens é a distribuição Linux **Debian**, que embora quando comparado com o windows ou outras distribuições Linux é considerado muito leve, para apenas aplicações em container como é nosso caso ainda existe uma opção melhor, o **Alpine**, o Debian possui muitos utilitários que dificilmente serão usados, enquanto o alpine é uma distribuição extremamente minimalista que contempla grande parte dos recursos necessários, vamos experimentar isso na prática. Devido ao Alpine ser extremamente minimalista, isso também dificulta para ataques mal intencionados ao nosso sistema, uma vez que o hacker tem recursos limitados para trabalhar isso dificulta seu trabalho.

5.1 Aplicando Alpine

Na prática anterior utilizamos as seguintes imagens:

- FROM mcr.microsoft.com/dotnet/sdk:9.0
- FROM mcr.microsoft.com/dotnet/aspnet:9.0

Agora vamos utilizar:

- FROM mcr.microsoft.com/dotnet/sdk:9.0-alpine
- FROM mcr.microsoft.com/dotnet/aspnet:9.0-alpine

```
# Etapa 1
FROM mcr.microsoft.com/dotnet/sdk:9.0-alpine AS build
WORKDIR /app

COPY ./CODIGO/*.csproj ./
RUN dotnet restore
COPY ./CODIGO ./

RUN dotnet publish -c Release -o /app/publish

# Etapa 2
FROM mcr.microsoft.com/dotnet/aspnet:9.0-alpine

WORKDIR /app
COPY --from=build /app/publish ./

ENTRYPOINT ["dotnet", "CODIGO.dll"]
```

Que resultará na seguinte imagem:

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
codigootimizado:latest	d80c8a10d557	329MB	92.9MB	
codigopesado:latest	ec3b3395e8a2	1.2GB	318MB	
codigoultraotimizado:latest	3cf075b1be59	167MB	50.9MB	

5.2 DockerHub

Publique isso no seu repositório do **DockerHub** como a :v3

6 Rootless

Voltando sobre a questão de segurança no alpine brevemente comentada a cima, graças aos pouco recursos nosso sistema está mais seguro, porém por padrão nosso sistema está rodando como "**root**", um usuário que tem acesso e permissão a todo o sistema, para aumentarmos a segurança do nosso sistema podemos forçar ele a rodar como um usuário limitado dentro do nosso container, com apenas permissões de um usuário.

Só precisamos adicionar a seguinte linha dentro do nosso Dockerfile, deste comando para baixo tudo irá rodar em rootless, ou seja, como um usuário comum:

```
RUN addgroup -S <NOME_DO_GRUPO> && adduser -S <NOME_DO_USUARIO> -G <NOME_DO_GRUPO>
```

****IMPORTANTE** - O conceito se mantém para todas as imagens, porém a maneira de fazer pode variar dependendo do **SO** base da imagem utilizada

Também vamos precisar alterar um pouco nosso **COPY**, que copia os arquivos executáveis do nosso sistema para o estágio final, precisamos adicionar a flag "--chown=<USUARIO>:<GRUPO>".

O comando chown serve para "**CH**ange **OWN**er", para assim podermos acessar nossos arquivos dentro da pasta /app

```
COPY --from=build --chown=<USUARIO>:<GRUPO> /app/publish ./
```

Resultado final:

```
# Etapa 1
FROM mcr.microsoft.com/dotnet/sdk:9.0-alpine AS build
WORKDIR /app

COPY ./CODIGO/*.csproj ./
RUN dotnet restore
COPY ./CODIGO ./

RUN dotnet publish -c Release -o /app/publish

# Etapa 2
FROM mcr.microsoft.com/dotnet/aspnet:9.0-alpine

RUN addgroup -S grupao && adduser -S usuariozao -G grupao

WORKDIR /app
COPY --from=build --chown=usuariozao:grupao /app/publish ./

USER usuariozao
```



```
ENTRYPOINT ["dotnet", "CODIGO.dll"]
```

7 Atividade

Agora que já sabemos como montar Dockerfiles corretos, aplique montando um compose para o projeto **Iduca**, as informações necessárias serão dadas a vocês, aplique o que foi passado nas aulas até então (incluindo **NGIX**).